Team Plan: Incident Zero (AI On-Call Engineer)

Bosch hackathon at PES University. Team of 5 (M1 to M5). Share this doc with the team, comment on anything unclear, and tick off the checklist at the end.

1. Goal and success criteria

Build an agent that takes an alert, investigates with tools, diagnoses with real evidence and a confidence level, proposes an allowlisted fix, waits for human approval, acts, verifies recovery, and writes a report.

We succeed if:

- The diagnosis is correct on practice incidents and on unseen ones (the final test uses unseen incidents and combinations).
- Every piece of evidence cited actually exists in the data.
- Nothing changes without approval, and the agent escalates when unsure.
- All five deliverables are in the repo by the code freeze.

2. Device setup

Device	Mode	Command	Owner	Purpose
Device A (16 GB)	Full	`make start`	M2	Integration laptop: faultctl, flag UI, final runs, hidden test, demo
Device B (8 GB)	Minimal	`make start-minimal`	M5	Second live stack for testing actions, and as backup
Other 3 laptops	Not running	Repo + offline bundles	M1, M3, M4	Build and test against recorded telemetry

- Everyone has Docker, Python 3.11 and Git verified, and a git email linked to their GitHub account.
- Device A and Device B run `docker compose ps` and `docker stats --no-stream` before the day to confirm they are healthy.
- Copy the offline incident bundles from the prep pack or USB drives onto every laptop.
- Do not share one running stack over Wi-Fi without asking a mentor first. The rule says everything runs on your own laptop.

3. Architecture and interfaces

These contracts let five people work in parallel without blocking each other. Agree on them in the first hour.

Layers

1. **Data adapter:** `get_logs()`, `get_traces()`, `get_metrics()`, `get_deploys()`, `get_container_health()`. Two backends, live and offline, switched by one config flag.
2. **Tools:** thin wrappers over the adapter that trim output and return structured results with source IDs (log line IDs, trace IDs, metric names and timestamps).
3. **Agent loop:** the LLM picks the next tool from what it has found so far, with a step limit and a token budget.
4. **Diagnosis:** structured output with component, cause, confidence and a list of evidence items. The validator checks that each item exists in the data.
5. **Action layer:** proposal, approval gate, dry-run, kill switch, Action API client, action log.
6. **Verification and report:** health check after the action, then rollback or escalate, then the report in the provided JSON schema plus a readable form.

Contracts to agree on early

- Tool result shape: `{source, id, timestamp, content}` so evidence can be validated later.
- Diagnosis shape: follows the organisers' JSON schema (M4 owns it, everyone reads it).
- Proposal shape: `{action, target, reason, dry_run_result}`.
- One config file for mode (live or offline), model name, confidence threshold and step limit.

4. Repo structure and git workflow

```
/agent       (loop, prompts)          M3
/tools       (telemetry tools)        M1, M2
/diagnosis   (validator, schema)      M4
/actions     (API client, approval)   M5
/reports     (generated reports)      M4
/docs        (architecture, eval)
README.md
```

- One person forks the template, and everyone confirms push access in Phase 0.
- Each member works in their own folder to avoid merge conflicts, using short-lived branches merged to `main`.
- Commit small and often (at least every 30 to 45 minutes) with clear messages.

- No hard-coded incident logic anywhere, and no API keys in the repo (use environment variables).
- Device A pulls `main` at each checkpoint to verify the integrated run.

5. Roles

Member	Owns	Device
M1	Log and trace tools, verification step	Offline bundles
M2	Metrics, container health, deploy history, alert trigger, faultctl, rollback path	Device A (full demo)
M3	Agent loop, prompts, remediation proposal, confidence rule, evaluation note	Offline bundles
M4	Evidence validation, JSON schema output, incident reports	Offline bundles
M5	Action API client, approval gate, dry-run, kill switch, action log, architecture page	Device B (minimal demo)

6. Phases with outputs and done criteria

Phase 0: Setup (Day 1, 12:00 to 13:00), Checkpoint 0

Member	Task	Done when
M1	Demo up with load; confirm traces and logs are visible (use the offline bundle if not running the demo)	Can view traces and logs for a request or incident
M2	faultctl injects and resets one incident	Metrics visibly change, then recover after reset
M3	LLM key works; minimal tool-calling hello-world	Model calls a dummy tool and uses its result
M4	Fork template, read JSON schema, create folder structure	Repo structure pushed
M5	Read Action API docs; one harmless dry-run call	Call succeeds and appears in the action log

Everyone commits one small file so all five are on record.

Phase 1: Agent can see (13:00 to 15:30), Checkpoint 1: Eyes

Member	Task	Done when
M1	`query_logs`, `query_traces` with trimmed, structured output	Returns relevant rows for a service and time window
M2	`query_metrics`, `container_health`, `deploy_history`, alert trigger	Alert fires from an error-rate or latency threshold
M3	Agent loop v1 with step limit	Agent picks tools sequentially on one incident
M4	Symptom summariser and incident data model	Raw telemetry becomes a short summary
M5	One-page architecture sketch; Action API client stub	Sketch exists; stub imports cleanly

Checkpoint goal: live run on one practice incident, plus the sketch.

Phase 2: Agent can diagnose (15:30 to 18:30), Checkpoint 2: Brain

Member	Task	Done when
M1	Test and fix application faults (payment, cart, catalog, memory leak, ad CPU/GC, traffic flood)	Correct component and cause on each
M2	Test and fix infrastructure faults (container down, cache/DB down, latency, CPU starvation)	Correct component and cause on each
M3	Prompt tuning; escalate-below-threshold rule	Low-confidence cases escalate instead of acting
M4	Evidence validator; schema-compliant output	Invented evidence is rejected; output validates against the schema
M5	Harness that runs all ten practice incidents and logs pass/fail	One command produces a results table

Checkpoint goal: correct diagnosis on at least 3 practice incidents, in the JSON schema. Unseen incidents decide the final score, so avoid per-incident rules.

Phase 3: Agent can act safely (Day 2, 09:30 to 12:00), Checkpoint 3: Hands

Member	Task	Done when
M1	Verification step: health metrics after the action	Reports recovered or not, with numbers
M2	Rollback and escalate path	Failed verification triggers rollback or escalation
M3	Remediation proposal from the allowlisted actions, with reason	Proposal always maps to an allowed action
M4	Incident report generator (timeline, root cause, evidence, action, prevention)	JSON plus readable report for each run
M5	Approval gate, dry-run, kill switch, action log	Nothing executes without a yes; kill switch halts a run

Checkpoint goal: one full run from alert to verified recovery, with the action log. Final end-to-end runs happen on Device A.

Phase 4: Harden and document (12:00 to 13:30), no new features

Member	Task
M1	README: setup and how to run
M2	Clean run from scratch on a fresh clone; bug fixes
M3	Evaluation note: which incidents were right or wrong and why, honestly
M4	Regenerate incident reports for all practice incidents
M5	Architecture overview page; README sections on approval gate, kill switch and known limitations

Everyone fills in their entry on the team form (what they built, tools and AI models used, including coding assistants).

Phase 5: Final verification, hidden test, demo

Member	Task
M1	Check the submission against all five deliverables before the freeze
M2	Hidden-test standby: watch logs, restart the stack if needed
M3	Demo presenter: drives the live run
M4	Q&A lead on diagnosis and evidence
M5	Q&A lead on safety and approvals

Everyone must be able to explain and change their own code (Rule 2), so each person rehearses a one-minute explanation of their component.

Stretch goals (only if Phase 3 is solid)

- Calibrated confidence: M3 and M4
- Chat-style approval UI: M5
- Incident memory: M1 and M2
- Reviewer agent and prevention suggestions: whoever is free after Phase 4

7. Rules everyone must follow

1. All code is written on the day from 09:15; no agent commits before then.
2. All members must commit code.
3. Change the system only through the Action API, and only after human approval.
4. Never read feature-flag settings to diagnose. Do not tamper with the demo, faultctl, hidden incidents or the scoring harness (disqualification).
5. No hard-coded answers.
6. Be honest about limitations; hiding a failure loses points.
7. When unsure, escalate with findings. It scores better than a confident wrong fix.

8. Risks and mitigations

Risk	Mitigation
Demo too heavy on an 8 GB laptop	Only two devices run it; others use offline bundles
Adapter works offline but breaks live	Test live mode on Device A at every checkpoint
LLM invents evidence	Validator rejects anything not found in the data
Overfitting to practice incidents	Test across all ten, including infrastructure faults and combinations
LLM spend limit hit	Trim tool outputs, cap steps and tokens, avoid pasting full logs
Merge conflicts	One folder per owner, small frequent commits
Last-minute breakage	Phase 4 is hardening only; fresh-clone run before the freeze
Someone blocked	Ask a mentor early (mentors help unblock but do not write code)

9. Communication

- A shared chat group, plus a 10-minute sync at the start of each phase.
- Each phase ends with a 5-minute integration check on Device A: pull `main`, run, note what failed.
- Raise blockers immediately, not at the checkpoint.

10. Deliverables checklist (by the code freeze)

- [ ] Source code and README (setup, run, approval gate, kill switch, limitations)
- [ ] One-page architecture overview
- [ ] Incident reports (JSON schema plus readable form)
- [ ] Evaluation note
- [ ] Team form

11. Pre-event checklist (per person)

- [ ] Docker with Compose v2 and Python 3.11 verified

- [ ] Git name and email set, email linked to GitHub
- [ ] Demo cloned and images pulled (Device A and B only)
- [ ] Offline incident bundles copied
- [ ] LLM key tested and stored as an environment variable
- [ ] 2-page LLM tool-calling primer read
- [ ] Laptop charged, charger packed

12. Questions to ask the organisers at kickoff

Team size is settled at 5. Still open:

- Is the event one day or two? (The brief says one day, the flow slide shows two.)
- Does code start at 09:15, or after the 11:30 kickoff on Day 1?
- Is the freeze 16:30, or the Day 2 13:30 to 14:00 submission slot?
- Is sharing one running stack over Wi-Fi allowed?
- Does minimal mode include the flag UI? (Check on first run.)